



American International University- Bangladesh
Department of Electrical and Electronic Engineering
EEE 4103: Microprocessor and Embedded Systems Laboratory

Title: Familiarization with the Timers of an Arduino Microcontroller Board, the study of LED blink test, and implementation of a simple traffic control system using its Timer0 function.

Introduction:

The objectives of this experiment are to-

1. Study the Timers of an Arduino Microcontroller Board.
2. Learn basic programming commands of the Timer functions.
3. Apply the coding techniques of the Timer functions.
4. Implement the LED blink test using Timer0 of an Arduino Microcontroller Board.
5. Implement a traffic light control system using an Arduino Microcontroller Board.

Theory and Methodology:

Every electronic component of a sequential logic circuit works on a time base. This time base helps to keep all the work synchronized. Without a time base, devices would have no idea as to when to execute a particular action or task. Thus, the timer is an important concept in the field of electronics.

A timer/counter is a piece of hardware built into the Arduino controller. It is like a clock and can be used to measure time events. A timer is a fixed-width register whose value increases/decreases automatically with the clock signal.

In AVR, timers are of two types, such as 8-bit and 16-bit timers. In an 8-bit timer, the register used is 8-bit wide whereas, in a 16-bit timer, the register width is 16 bits. This means that the 8-bit timer can count $2^8 = 256$ steps starting from 0 to 255. Similarly, a 16-bit timer can count $2^{16} = 65536$ steps starting from 0 to 65535.

Experimental Setup:

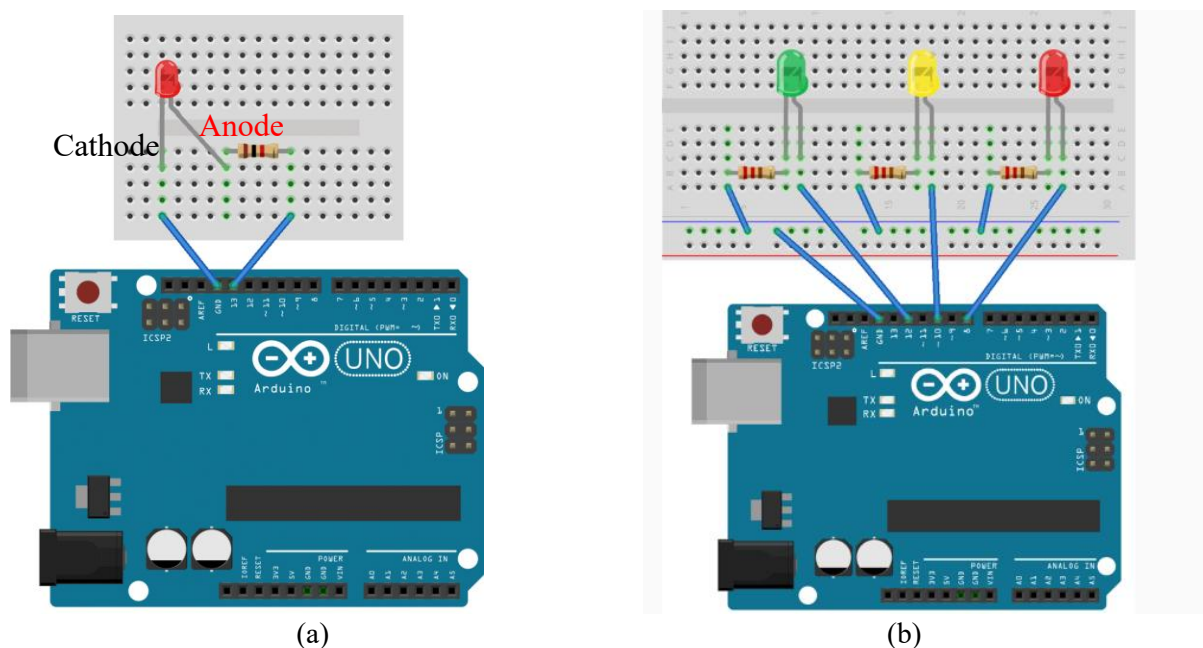


Fig. 1 Experimental Setup of a Traffic Control System using an Arduino Microcontroller Board.

Apparatus:

- 1) Arduino IDE (2.0.1 or any recent version)
- 2) Arduino Microcontroller board
- 3) LED lights (Red, Green, and Yellow)
- 4) Three 100 Ω resistors
- 5) Jumper wires

Experimental Procedure:

The main task of this experiment is to understand and implement an LED blinking and traffic light control system. Connect the circuits as per the diagram of Fig. 1. Then plug in the Arduino microcontroller board to the PC.

Using Arduino IDE to write code:

1. Open the Arduino Uno IDE 2.0.1 (version may have been updated automatically on your computer) and a blank sketch will open. The window as in Fig. 2 will come up on your PC:

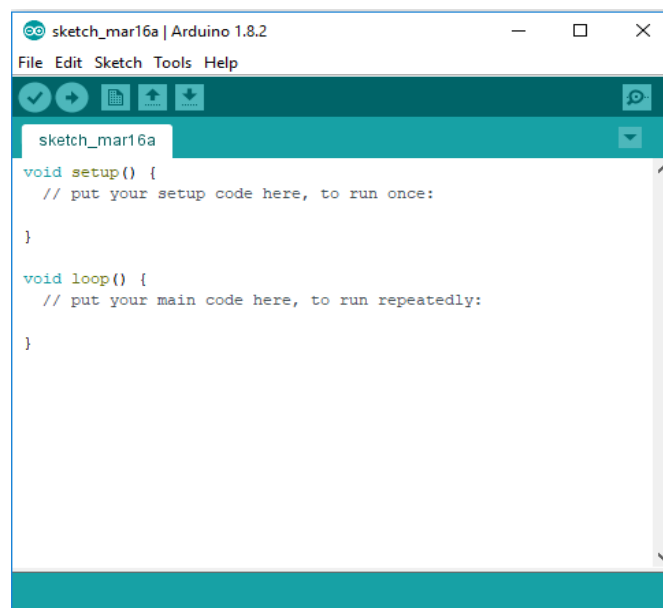


Fig. 2 IDE Environment (text editor)

2. Now, write the following program on the blank sketch for the LED blink test.

```
// Pin numbers are defined by the corresponding LED colors
#define YELLOW_PIN 10

// The delay amounts are declared in ms; 1 s = 1000 ms
int red_on = 2000;

// Declaring the timer function to count 1 ms of delay instead of calling the delay function
int delay_timer (int milliseconds){
    int count = 0;
    while(1)
    {
        if(TCNT0 >= 15) // Checking if 1 millisecond has passed
        {
```

The number of counts
needed to reach 1 ms =
(1,000 μ s/64 μ s) – 1 =
15.625 - 1 $\approx$ 15.

```

    TCNT0=0;
    count++;
    if (count == milliseconds) //checking if required milliseconds delay has passed
    {
        count=0;
        break; // exits the loop
    }
}
return 0;
}

void setup() {
    //define pins connected to LEDs as outputs
    pinMode(RED_PIN, OUTPUT);

    //set up the timer using timer registers
    TCCR0A = 0b00000000;
    TCCR0B = 0b00000101; //setting pre-scaler for timer clock
    TCNT0 = 0;
}

void loop() {
    //to make the red LED turned on
    digitalWrite(RED_PIN, HIGH);
    delay_timer(red_on);

    //turning off the red LED
    digitalWrite(RED_PIN, LOW);
    delay_timer(red_on);
}

```

Code of a traffic light control system with Timer0:

1. Now, open another blank sketch in the Arduino IDE and copy-paste the following program to implement the traffic control system using Timer0.

```

// Pin numbers are defined by the corresponding LED colors
#define RED_PIN 8
#define YELLOW_PIN 10
#define GREEN_PIN 12

//Define the delays for each traffic light color
int red_on = 6000; //3 s delay
int green_on = 3000; //3 s delay
int yellow_blink = 500; //0.5 s delay

```

```
// Declaring the timer function to count 1 ms of delay instead of calling the delay function
int delay_timer (int milliseconds){
    int count = 0;
    while(1)
    {
        if(TCNT0 >= 15) // Checking if 1 millisecond has passed
        {
            TCNT0 = 0;
            count++;
            if (count == milliseconds) //checking if required milliseconds delay has passed
            {
                count = 0;
                break; // exits the loop
            }
        }
    }
    return 0;
}
```

The number of counts needed to reach 1 ms = $(1,000 \mu\text{s}/64 \mu\text{s}) - 1 = 15.625 - 1 \approx 15$.

```
void setup() {
    //Define pins connected to LEDs as outputs
    pinMode(RED_PIN, OUTPUT);
    pinMode(YELLOW_PIN, OUTPUT);
    pinMode(GREEN_PIN, OUTPUT);

    //Set up the Timer0
    TCCR0A = 0b00000000;
    TCCR0B = 0b00000101; //setting pre-scaler for timer clock
    TCNT0 = 0;
}
```

```
void loop() {
    //to make the green LED turned on
    digitalWrite(GREEN_PIN, HIGH);
    delay_timer(green_on);
    //to make the green LED turned off
    digitalWrite(GREEN_PIN, LOW);

    //for turning the yellow LED on and off for 4 times
    for(int i = 0; i < 4; i = i+1)
    {
        digitalWrite(YELLOW_PIN, HIGH);
        delay_timer(yellow_blink);
        digitalWrite(YELLOW_PIN, LOW);
        delay_timer(yellow_blink);
    }

    //to make the red LED turned on
    digitalWrite(RED_PIN, HIGH);
    delay_timer(red_on);
    //to make the red LED turned off
    digitalWrite(RED_PIN, LOW);
}
```

2. After writing the program you must save your code file in a folder. For this purpose, please go to the File Menu → Save As → give a File name in the file name box (for example, Traffic_light_control) → Select Save.

N.B. After saving your code, a sketch file with the desired file name will be stored in a folder with the same file name.

3. Now you need to verify/compile your code to find out and correct the errors. For this, please go to the Sketch Menu → Verify/Compile or press Ctrl+R or click on this button  just below the menu bar. If the code is correct then the code compilation done message will be displayed, otherwise, an error message will be displayed.
4. After the compilation is done successfully, you need to upload your code onto the Arduino Uno board. To upload the program, connect your Arduino Uno R3 board to your PC with a USB cable. Before uploading the code, select the board type and port at your Arduino IDE. For this purpose, please go to the Tools Menu → Board: “Arduino Uno” → Arduino Uno. Then again go to the Tools Menu → Ports → COMx, here x means an integer number that corresponds to the Arduino Uno board that is automatically detected by the IDE.
5. After you have selected the board and port, you must upload the code. For this, please go to the Sketch Menu → Upload or press Ctrl+U or click on this button  just below the menu bar. If the code is correct then the code upload-done message will be displayed, otherwise, an error message will be displayed. Based on the error message (if any), please go to the code, and correct them. Usually, students forget to set up the Tools or select an inappropriate board or COMx port number to get such kinds of error messages.

Questions for Report Writing:

- 1) Include all codes and scripts in the lab report following the lab report writing template.
- 2) Show the output/results in the form of images. Give their captions and descriptions.
- 3) Configure the system to have delays for outputs according to your ID. Consider the last three digits from your ID (if your ID is XY-PQABC-Z then consider A s delay for the RED LED, B s delay for the YELLOW LED, and C s delay for GREEN LED). In case any digit is zero then use Z s delay by default. Include the program and results within your lab report.
- 4) Draw the **program flow chart** of both codes and explain them.
- 5) Include the **Proteus simulation** of the LED blink program and traffic light control system. Explain the simulation methodology. You may learn the simulation from the following video link: <https://www.youtube.com/watch?v=yHB5it0s2oU>

Reference(s):

- [1] <https://www.arduino.cc/>.
- [2] ATmega328 manual
- [3] <https://www.avrfreaks.net/forum/tut-c-newbies-guide-avr-timers>
- [4] <https://maxembedded.com/2011/06/avr-timers-timer0/>